

# Wavelength

Self-Hosted Music Streaming Server

Project Architecture & Technical Documentation

Generated July 2026

# Table of Contents

---

1. Project Overview
2. Project Structure
3. Backend (Crystal)
  1. Entry Point & Bootstrapping
  2. Configuration (AppConfig)
  3. Library Scanner & Track Model
  4. Transcoding Service
  5. Action System & Routing
  6. API Endpoints
  7. Audio Streaming
4. Frontend
  1. Architecture Overview
  2. Component Breakdown
  3. Audio Playback & Visualizer
5. Data Flow
6. External Dependencies
7. Deployment

# 1. Project Overview

---

**Wavelength** is a self-hosted, browser-based music streaming server written in [Crystal](#) using the [Kemal](#) web framework. It scans a local music library on disk, presents it through a web UI, and streams audio directly to any browser with support for mobile background playback via the Media Session API.

Key characteristics:

- **No database** — everything is in-memory; tracks are re-scanned from disk periodically.
- **No build tools on the frontend** — vanilla JavaScript (ES modules), plain CSS, no npm/bundler.
- **Transparent transcoding** — unsupported codecs (eac3, truehd, ac3, dts) are converted to MP3 on-the-fly or asynchronously.
- **Docker-first** — multi-stage Dockerfile, runs as non-root user.

## 2. Project Structure

---

```

wavelength/
  â src/
    â main.cr # Entry
point
  â configuration/
    í app_config.cr # YAML/
env config
  â classes/
    â scanner.cr # Track
struct + library scanner
  í progress_tracker.cr # In-
memory transcode progress
  â service/
    í transcode_service.cr # Async
ffmpeg transcoding
  â actions/
    â configuration/
      í register_actions.cr #
Compile-time route registration
  â base/
    â base_action.cr # Abstract
base (content-type, streaming)
  â content_type.cr #
ContentType enum + MIME mapping
  í track_action.cr # Track
lookup + path resolution
  â home_page_action.cr # GET /
  â static_action.cr # GET /*
(whitelist-gated)
  í rest_api/
    â tracks_action.cr # GET /
api/tracks

```

```
        â  audio_action.cr          # GET /
audio/:id
        â  rescan_action.cr         # POST /
api/rescan
        í  transcode_action.cr      # POST/GET
/api/transcode/:id
    â  views/
        í  home_page.ecr           # ECR
template (SPA shell)
    í  public/
        â  js/                      # App.js,
Player.js, TrackList.js, ...
        í  css/                    # base.css,
player.css, ...
    â  config/
        í  config.yml
    í  Dockerfile
```

---

## 3. Backend (Crystal)

---

### 3.1 Entry Point & Bootstrapping

The application starts in `src/main.cr`. The boot sequence is:

1. Require Kemal and all source files via `glob requires`.
2. Load configuration: `AppConfig.load("config/config.yml")`.
3. Start the library scanner:  
`Scanner.start_periodic_scan`.
4. Auto-register all routes via the `register_actions` compile-time macro.
5. Start the Kemal HTTP server on `host:port`.

### Route Registration Macro

Instead of manually listing routes, the `register_actions` macro uses Crystal's compile-time metaprogramming to iterate over all subclasses of `BaseAction`, find methods annotated with `@[Get("/path")]` or `@[Post("/path")]`, and register them as Kemal route handlers. Adding a new route is as simple as creating a new subclass with an annotated method.

## 3.2 Configuration (AppConfig)

The AppConfig module (`src/configuration/app_config.cr`) is a singleton that holds all runtime configuration. It parses a YAML config file and allows environment variables to override file values.

| Constant / Setting   | Default                                     | Description                           |
|----------------------|---------------------------------------------|---------------------------------------|
| CACHE_DIR            | /tmp/<br>wavelength                         | Transcode<br>cache<br>directory       |
| UNSUPPORTED_CODECS   | {eac3,<br>truehd,<br>ac3, dts}              | Codecs that<br>require<br>transcoding |
| SUPPORTED_EXTENSIONS | {.mp3,<br>.wma,<br>.m4a,<br>.ogg,<br>.flac} | Scanned<br>audio<br>formats           |
| host                 | 0.0.0.0                                     | Bind<br>address                       |
| music_path           | /music                                      | Music<br>library root                 |

|                            |            |                 |
|----------------------------|------------|-----------------|
| <code>port</code>          | 3000       | HTTP port       |
| <code>scan_interval</code> | 60 seconds | Rescan interval |

### 3.3 Library Scanner & Track Model

#### Track Struct

Defined in `src/classes/scanner.cr`, the `Track` struct is JSON-serializable and represents a single audio file. Key fields:

| Field                             | Type   | Description                                                          |
|-----------------------------------|--------|----------------------------------------------------------------------|
| <code>id</code>                   | String | Relative path with non-alphanumeric chars replaced by <code>_</code> |
| <code>title, artist, album</code> | String | Metadata from <code>ffprobe</code> or filename parsing               |



|                                |                      |                               |
|--------------------------------|----------------------|-------------------------------|
| <code>filename</code>          | <code>String</code>  | Relative path from music root |
| <code>codec</code>             | <code>String</code>  | Audio codec (e.g. aac, flac)  |
| <code>duration</code>          | <code>Float64</code> | Length in seconds             |
| <code>needs_transcoding</code> | <code>Bool</code>    | True if codec is unsupported  |
| <code>transcoded</code>        | <code>Bool</code>    | True if cached MP3 exists     |

## Scanner Module

The `Scanner` module manages an in-memory array of `Track` objects, protected by a mutex for thread safety.

- **`scan_music_directory`** — The core scan method. Recursively globs the music directory, filters by supported extensions, runs `ffprobe` on each file to extract metadata (codec, duration, ID3/Vorbis/MP4 tags), falls back to filename parsing if tags are unavailable, and sorts results by artist/album/title.

- **start\_periodic\_scan** — Performs an immediate scan, then spawns a background fiber that re-scans every `scan_interval` seconds.
- **all\_with\_transcode\_status** — Returns all tracks with live transcoding status by checking whether the cached MP3 file exists on disk.
- **cache\_path\_for(filename)** — Computes the cache file path under `CACHE_DIR` for a given relative filename.

## Metadata Extraction (ffprobe)

The `probe_metadata` method spawns `ffprobe` as a subprocess with `-print_format json -show_format -show_streams`, then parses the JSON output to extract:

- Audio codec from the first audio stream (`codec_name`).
- Duration from `format.duration`.
- Tags from `format.tags`, supporting multiple naming conventions: Vorbis (`title/artist/album`), ID3v2 (`TIT2/TPE1/TALB`), and iTunes (`©nam/©ART/©alb`).

## 3.4 Transcoding Service

`TranscodeService (src/service/transcode_service.cr)` handles asynchronous audio

transcoding via `ffmpeg`.

- **Concurrency control:** A `Channel(1)` semaphore limits to 3 concurrent `ffmpeg` processes.
- **Progress tracking:** Polls the output file size every 300ms, estimating progress as `(current_size / (duration × 40000)) × 100`, capped at 99% until completion.
- **Abort handling:** If a new request arrives for an already-running ID, the old process is terminated.
- **Output:** Transcodes to MP3 at 320kbps (`libmp3lame`) into a temp file, then atomically renames to the final cache path on success.

## 3.5 Action System & Routing

The backend uses an action-based architecture. All routes are classes that inherit from `BaseAction` (or its subclass `TrackAction`).

### Class Hierarchy

```
BaseAction
|-- setContentType(response, type)
|-- stream_file(response, path, offset, length)
|
+-- TrackAction
```

```

|           |-- find_track_or_404(http_context)
|           |-- resolve_track_path(track)
|
+-- HomePageAction          (GET /)
+-- StaticAction            (GET /*)
+-- TracksAction            (GET /api/tracks)
+-- AudioAction             (GET /audio/:id)
+-- RescanAction            (POST /api/rescan)
+-- TranscodeAction         (POST/GET /api/transcode/:id)

```

## BaseAction

Provides shared constants (`Html`, `Json`, `Js`, `Css` content type aliases), the `setContentType` helper, and the `stream_file` method for buffered file streaming with a 64KB buffer.

## TrackAction

Extends `BaseAction` with track-specific helpers:

- `find_track_or_404` — Extracts the `:id` URL parameter, looks up the track, returns 404 if not found.
- `resolve_track_path` — Resolves a track's relative filename to an absolute path and validates it stays within `music_path` (path traversal prevention).

## 3.6 API Endpoints

| Method | Path               | Action          | Description                        |
|--------|--------------------|-----------------|------------------------------------|
| GET    | /                  | HomePageAction  | Serves the SPA HTML shell          |
| GET    | /*path             | StaticAction    | Serves .js/.css files from public/ |
| GET    | /api/tracks        | TracksAction    | Returns all tracks as JSON         |
| GET    | /audio/:id         | AudioAction     | Streams audio with Range support   |
| POST   | /api/rescan        | RescanAction    | Triggers library rescan            |
| POST   | /api/transcode/:id | TranscodeAction | Starts async transcode             |

|     |                                                    |                 |                                   |
|-----|----------------------------------------------------|-----------------|-----------------------------------|
| GET | <code>/api/<br/>transcode/:id/<br/>progress</code> | TranscodeAction | Returns<br>progress<br>percentage |
|-----|----------------------------------------------------|-----------------|-----------------------------------|

## 3.7 Audio Streaming

The `AudioAction` (`GET /audio/:id`) is the core streaming endpoint:

1. Finds the track and resolves its file path (with path traversal guard).
2. If the codec is unsupported, synchronously transcodes to MP3 via `ensure_transcoded` and serves the cached file.
3. If the codec is natively supported, serves the original file directly.
4. Supports **HTTP Range requests** for seeking/scrubbing:
  - Returns `206 Partial Content` with correct `Content-Range`, `Content-Length`, and `Accept-Ranges` headers.
  - Uses the `stream_file` helper for efficient buffered I/O.

MIME type mapping:

| Extension | Content-Type   |
|-----------|----------------|
| .mp3      | audio/mpeg     |
| .wma      | audio/x-ms-wma |
| .m4a      | audio/mp4      |
| .ogg      | audio/ogg      |
| .flac     | audio/flac     |

# 4. Frontend

---

## 4.1 Architecture Overview

The frontend is built with **vanilla JavaScript** using ES modules, with no frameworks or build tools. CSS is loaded dynamically at runtime using the `adoptedStyleSheets` API. The HTML shell is a Crystal ECR template (`home_page.ecr`).

### Module Map

| Module               | File        | Responsibility                                                                                                   |
|----------------------|-------------|------------------------------------------------------------------------------------------------------------------|
| <code>main.js</code> | Entry point | Instantiates App, calls <code>loadTracks()</code>                                                                |
| <code>App.js</code>  | Controller  | Wires Player, TrackList, Visualizer; handles UI events, keyboard shortcuts, Media Session API, transcode polling |



|                            |                 |                                                                                                                          |
|----------------------------|-----------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>Player.js</code>     | Playback engine | Manages the <code>&lt;audio&gt;</code> element, playlist, play/pause/next/prev, shuffle/repeat, transcode-then-play flow |
| <code>TrackList.js</code>  | List renderer   | Manages full and filtered track lists, search/filter, grouping by artist/album with sticky headers                       |
| <code>TrackRow.js</code>   | Row component   | Builds a single track row DOM element, active state, transcode progress bar                                              |
| <code>Visualizer.js</code> | Visualizer      | Web Audio API frequency visualizer (64 bars, rainbow HSL coloring)                                                       |
| <code>dom.js</code>        | DOM utilities   | <code>h()</code> virtual-element builder (XSS-safe), <code>adoptStylesheet()</code>                                      |

## 4.2 Component Breakdown

## App.js

The central controller that:

- Initializes all components on DOM ready.
- Fetches `/api/tracks` and passes the data to `TrackList`.
- Binds event handlers: seek bar, volume slider, keyboard shortcuts (space, arrow keys), rescan button, search input.
- Manages transcode progress polling: when a track needs transcoding, polls `/api/transcode/:id/progress` every 500ms until complete.
- Integrates with the Media Session API for lock-screen / notification controls on mobile.
- Updates the now-playing display (title, artist) and document title on track change.

## Player.js

Manages the HTML5 `<audio>` element:

- `playIndex(idx)` — Plays a track by index. If the track needs transcoding, triggers `POST /api/transcode/:id` first, then sets `audio.src` once cached.
- Supports **shuffle** (randomized playlist order) and

**repeat** (single track or full playlist loop) modes.

- Auto-advances to the next track on `ended` event.

## TrackList.js

Renders the scrollable track list:

- Groups tracks by artist and/or album with sticky group headers.
- Supports text search/filter across title, artist, and album fields.
- Delegates individual row rendering to `TrackRow.js`.

## dom.js — The `h()` Helper

A minimal virtual-element builder that creates DOM elements safely. Text content is set via `textContent` (never `innerHTML`), preventing XSS when rendering user-provided track metadata.

```
// Usage example:
h("div", { class: "track-row" }, [
  h("span", {}, ["Artist Name"]),
  h("span", {}, ["Track Title"]),
])
```

## 4.3 Audio Playback & Visualizer

## Playback Flow

1. User clicks a track row →

`TrackList.onSelect(idx) →`

`Player.playIndex(idx).`

2. Player checks `track.needs_transcoding:`

- If **yes**: sends `POST /api/transcode/:id`, polls progress, then sets `audio.src = /audio/:id` and plays.
- If **no**: directly sets `audio.src = /audio/:id` and plays.

5. Browser fetches `/audio/:id` → backend streams audio bytes with Range support.

6. On track end: `Player.next()` (or repeat if enabled).

## Visualizer

Uses the **Web Audio API** to draw a real-time frequency spectrum:

- Creates an `AudioContext` and connects the `<audio>` element via `MediaElementSource`.
  - Uses an `AnalyserNode` with `fftSize = 128` to get 64 frequency bins.
  - Draws bars on a `<canvas>` element with rainbow HSL coloring (hue mapped to bar index).
-

## 5. Data Flow

---

### Library Scan

```
Disk (music_path/**/*)
  |> Scanner.scan_music_directory()
  |> ffprobe extracts metadata per file
  |> Filename fallback for missing tags
  |> Creates Track structs, sorts by artist/album/title
  |> Stores in Scanner.@@tracks (mutex-protected)
```

### API Request

```
Browser: GET /api/tracks
  |> TracksAction
  |> Scanner.all_with_transcode_status()
  |> Returns JSON array of Track objects

Browser: GET /audio/:id
  |> AudioAction
  |> find_by_id + resolve_track_path (with traversal gu
  |> If unsupported codec: ensure_transcoded() then ser
  |> If supported: serve original file
  |> HTTP Range support for seeking
```

### Transcoding

1. Frontend: POST /api/transcode/:id
2. Backend: TranscodeService.start() spawns ffmpeg  
ffmpeg -i input -c:a libmp3lame -b:a 320k -f
3. Progress: File size polled every 300ms vs expected (du
4. Frontend: Polls GET /api/transcode/:id/progress every
5. Complete: tmp renamed to final cache path
6. Frontend: Sets audio.src = /audio/:id, serves cached M

## 6. External Dependencies

---

### Crystal Libraries (Shards)

| Dependency     | Purpose                                              |
|----------------|------------------------------------------------------|
| kemal          | HTTP web framework (built on Crystal's HTTP::Server) |
| radix          | Radix tree router (transitive Kemal dependency)      |
| exception_page | Developer error pages (transitive)                   |
| backtracer     | Backtrace formatting (transitive)                    |

### System Tools

| Tool                 | Usage                                         | Required?                                 |
|----------------------|-----------------------------------------------|-------------------------------------------|
| <code>ffprobe</code> | Audio metadata extraction during library scan | Optional (falls back to filename parsing) |
| <code>ffmpeg</code>  | Audio transcoding (eac3/truehd/ac3/dts → MP3) | Required only for unsupported codecs      |

## Browser APIs

| API                             | Purpose                                                  |
|---------------------------------|----------------------------------------------------------|
| Media Session API               | Lock-screen / notification controls on mobile            |
| Web Audio API                   | Frequency visualizer (AnalyserNode + MediaElementSource) |
| <code>adoptedStyleSheets</code> | Dynamic stylesheet loading                               |
| Fetch API                       | AJAX communication with the backend                      |

---

## 7. Deployment

---

Wavelength is designed to run as a Docker container using a multi-stage build:

- **Build stage:** Crystal 1.12 on Alpine compiles the application.
- **Runtime stage:** Alpine 3.19 with only the compiled binary and runtime dependencies.
- Runs as non-root user (`wavelength:wavelength`) for security.
- Music directory mounted read-only at `/music`.
- Config mounted read-only at `/app/config/config.yml`.
- Transcode cache at `/tmp/wavelength` (ephemeral, inside container).

```
# docker-compose.yml
services:
  wavelength:
    build: .
    ports:
      - "3000:3000"
    volumes:
      - ./music:/music:ro
      - ./config:/app/config:ro
```